

שנקר ביי"ס גבוה להנדסה ולעיצוב
החוג להנדסת תוכנה
קורס 'רשתות תקשורת' (קורס מספר 3503836)
תשפ"ו – סמסטר ב'

מרצה: יצחק נודלר

תרגיל בית מס' 5 – פתרון

השאלות בתרגיל זה מתייחסות לתסריט הבא:

מרצה, לצורך העניין אני, מריץ את Chrome.
במחשב desktop ששמו room247 ואשר יש לו כתובת IP מקומית.

המרצה מקליד את ה- URL הבא: online.abc.edu
זהו ה- host name של שרת Moodle אשר מנהל את הקורסים השונים במכללה.
השרת 'נמצא' ומנוהל 'בתוך' הרשת הארגונית של המכללה.

כעת, המרצה מעלה, upload, קובץ לאתר הקורס.
נניח כי הקורס הוא Computer Networks.
נניח כי שם הקובץ הוא L01.pdf
וכי הגודל שלו הוא 1.5MB

הערה:
המונח 'קרלי' [= kernel] בשאלות מתייחס לקרנל של מערכת ההפעלה.
כפי שלמדנו בהרצאות.
היינו למימוש של TCP 'בתוך' כלל הקוד של הקרנל.

שאלה מס' 1

א. הקובץ L01.pdf שגודלו 1.5MB נשלח מ-room247.
בהנחה שגודל ה- Receive Buffer בשרת הוא 64KB בלבד, תארו את הדינמיקה בין ה- send window ל- receive window.
כיצד TCP בצד השרת מעדכן את ה- Advertised Window [= ערך השדה Receive Window שב- TCP header] בסגמנטים של ה- ACK שהוא שולח ל- TCP בצד הלקוח?
ב. ומה קורה אם השרת חווה עומס עיבוד וה- Receive Buffer מתמלא?

ה- *receive window (rwnd)* מחושב כ: *ReceiveBuffer - (LastByteReceived - LastByteRead)*.
כאשר ה- *Buffer* מתמלא, השרת שולח ACK עם *window size* קטן יותר.
אם הוא מתמלא לחלוטין, השרת יכריז על *Zero Window*.
ה- *Sender* יעצור שליחה, אך ימשיך לשלוח "*Zero Window Probes*" כדי לגלות מתי השרת פינה מקום (ע"י קריאת הנתונים מה- *Buffer* ע"י האפליקציה).
זהו מנגנון ה- *Flow Control*.

שאלה מס' 2

א. תארו את המעבר ממרחב המשתמש (User Space) למרחב הליבה (Kernel Space) במהלך הקריאה `send(sockfd, buffer, 1500000, 0)`.
האם ה-Kernel ימתין עד שכל ה-1.5MB יועברו ל-NIC?
ב. פרטו את התפקיד של ה-Socket Send Buffer בתהליך זה.

הקריאה `send()` מעתיקה את ה-Data ממרחב המשתמש ל-Socket Send Buffer ב-Kernel. אם ה-Buffer אינו מספיק גדול, התהליך יחסם (Blocking) או יחזיר שגיאה (אם הוא Non-blocking). ה-Kernel מחלק את הנתונים ל-Segments לפי ה-MSS ומעביר אותם לשכבת ה-IP. התהליך לא ממתין ל-ACK עבור כל הסגמנטים כדי להמשיך, אלא ממשיך עד שה-Buffer מתמלא או שה-Data הועתק במלואו.

שאלה מס' 3

א. בחיבור שבין ה-Desktop לשרת המקומי, ה-Kernel מנהל מדידת RTT דינמית. מדוע לא מספיק להשתמש ב-Sample RTT יחיד עבור כל סגמנט?
ב. הסבירו את חשיבות הממוצע המשוקלל בחישוב ה-RTO $[RTO = Retransmission\ TimeOut]$, וכיצד הוא מתמודד עם תנודות קיצוניות ב-Variance של ה-RTT.

*שימוש ב-Sample בודד רגיש ל-"Spurious Timeouts" (זיהוי שגוי של איבוד חבילה עקב תנודות זמניות). האלגוריתם מחשב ממוצע נע משוקלל (SRTT) וסטיית תקן (RTTVAR). ה-RTO נקבע ע"י $SRTT + 4 * RTTVAR$. זה מבטיח שה-Timeout מספיק רחב כדי לספוג תנודות ב-Network Delay מבלי לבצע Retransmission מיותר.*

שאלה מס' 4

נתון כי סגמנט ה-SYN-ACK שחזר מהשרת מכיל $Sequence\ Number = X$ ו-Acknowledgment Number = Y.
א. מה הערכים הללו מייצגים ביחס ל-ISR של שני הצדדים?
ב. הסבירו את הקשר בין ה-Y ל-Sequence Number של ה-SYN המקורי ששלח הלקוח.

*X הוא ה-ISR של השרת.
Y הוא $Client_ISR + 1$.
ה-Acknowledgment Number מאשר את ה-SYN של הלקוח, ולכן הוא מצפה לקבל את הסגמנט הבא עם $Sequence\ Number\ Client_ISR + 1$.*

שאלה מס' 5

בתחילת העברת הקובץ L01.pdf, ה-Sender ב-room247 נמצא במצב Slow Start. א. מהו המשתנה cwnd (Congestion Window), וכיצד הוא משתנה בכל פעם שמתקבל ACK תקין? ב. מדוע השם "Slow Start" מטעה מבחינת קצב צמיחת ה-Window?

cwnd הוא ה-Window שה-Sender מעריך שיכול לעבור ברשת ללא גרימת עומס. במצב Slow Start, על כל ACK שמתקבל, ה-cwnd גדל ב-MSS 1 (עבור כל ACK של סגמנט יחיד).

מכיוון שה-Window מכפיל את עצמו בכל RTT (גידול מעריכי), השם "Slow Start" מטעה – הגידול הוא מהיר מאוד. ה-Kernel משתמש ב-ssthresh כדי לעבור ל-Congestion Avoidance ברגע שה-cwnd מגיע לסף מסוים.

שאלה מס' 6

נניח שה-MTU [Maximum Transfer Unit] של המסלול [route =] שבין מחשב הלקוח למחשב השרת הוא 1500 בתים. ערכו של ה-MSS בצד הלקוח הוא 1460 בתים. א. אם ה-Kernel שולח סגמנט עם Payload של 1460 בתים, כיצד הוספת כותרות ה-TCP וה-IP משפיעה על גודל החבילה הסופי? ב. האם ה-IP יכול לבצע Fragmentation במקרה זה?

1460.

$$(Data) + 20 (TCP Header) + 20 (IP Header) = 1500 \text{ bytes}$$

זה בדיוק ה-MTU שה-Kernel מנהל את ה-MSS כך שלא יהיה צורך ב-Fragmentation ע"י ה-IP, שכן Fragmentation פוגע בביצועים (אם fragment אחד אובד, כל החבילה המקורית צריכה להישלח שוב).

שאלה מס' 7

ב- kernel, ה-Data נשמר ב-sk_buff.
מדוע ה- Kernel מעדיף להעביר Pointer ל- Buffer בין שכבות ה- TCP/IP במקום להעתיק את ה- Data עצמו. היינו, להמנע מהצורך לבצע data copying.

א. מדוע?

ב. מה הקשר לביצועים?

*העתקת נתונים בין שכבות היא יקרה מבחינת CPU וזיכרון.
ה- skb מאפשר לכל שכבה (TCP, IP, Ethernet) להוסיף את ה- Header שלה בתחילת ה- Buffer (Prepend) ללא העתקת ה- Payload.
זהו יסוד ה- Zero-copy networking.*

שאלה מס' 8

א. בקריאת listen(sockfd, backlog), מהי המשמעות של ה- backlog?
ב. מה קורה ל- SYN segments שמגיעים בזמן שה- Queue מלא?

*ה- backlog מגדיר את מספר החיבורים הממתינים ל- accept().
אם ה- Queue מלא, ה- Kernel בדרך כלל יתעלם מ- SYN סגמנטים חדשים (Drop), מה שגורם ללקוח לקבל Connection Refused או Time-out.*